

Bono 1: Permutaciones y k-permutaciones

Diseñe un programa que reciba dos enteros n y r , con $0 \leq r \leq n$, y calcule el número de formas de ordenar r objetos distintos tomados de un conjunto de n objetos distintos.

El programa debe calcular:

$${}_nP_r = \frac{n!}{(n-r)!}$$

El programa debe permitir:

1. Calcular $n!$
2. Calcular ${}_nP_k$
3. Validar que n y r sean enteros no negativos
4. Mostrar el procedimiento usado
5. Comparar al menos dos casos, por ejemplo $10P3$ y $20P5$.

Extensión opcional: comparar una implementación recursiva y una iterativa del factorial.

Descripción matemática

Las permutaciones son una técnica de conteo para averiguar todas las formas en las que se puede reordenar un conjunto de n elementos. La fórmula para lograr esto es:

$$P(n) = n!$$

Esto solo aplica cuando todos los elementos del conjunto son indistinguibles y en fila. Si están en círculo usamos:

$$P_c(n) = (n-1)!$$

Y para elementos repetidos usamos combinación con repetición:

$$P_r(n) = \frac{n!}{n_1!n_2!\dots n_k!}$$

Donde $n_1, n_2, \dots$, son las repeticiones de cada elemento.

Por el otro lado, la k-permutación es un tipo de permutación en el que se seleccionan k elementos de un conjunto de n elementos y cuenta cuantas posibilidades de elegir hay. La fórmula para esto es:

$${}_nP_k = \frac{n!}{(n-k)!}$$

Es necesario que $k \leq n$, porque si no, $n - k$ es negativo, y por ende $(n - k)!$ no existe.

En la permutación el orden de selección de los elementos siempre importa. La permutación normal puede considerarse un caso especial de la k-permutación donde $k = n$.

Implementación

1. Permutaciones

En la forma más básica, la librería `math` de Python ofrece la función `math.factorial` que podemos usar directamente:

```
import math as m

print(m.factorial(7)) # -> 5040
```

Sin embargo vamos a crear dos alternativas, una iterativa y una recursiva. Ambas bien conocidas en programación. Usaremos anotaciones de tipo para mejor legibilidad y errores para manejo de forma externa.

```
## Calcula el factorial de un entero dado usando iteración
def iterative_factorial(n: int) -> int:
    # Error de valor si es menor que cero
    if n < 0:
        raise ValueError("Valor no válido")

    fact: int = 1
    for i in range(n):
        fact *= i + 1

    return fact

# También podríamos usar esta opción (mueve el import al inicio si quieres)
# import functools as f
# return f.reduce(lambda x, y: x * y, range(1, n + 1))

## Calcula el factorial de un entero dado usando recursión
def recursive_factorial(n: int) -> int:
    # Error de valor si es menor que cero
    if n < 0:
        raise ValueError("Valor no válido")

    # Caso base y recursivo (operador ternario)
    return n * recursive_factorial(n - 1) if n else 1
```

Este aprovecha una propiedad de los factoriales:

$$n! = n(n - 1)!$$

Si se repite este proceso con el factorial resultante de la derecha, podemos «desenrollar» un factorial hasta llegar a la forma base $0! = 1$. La forma recursiva desenrolla hasta llegar al caso base y procede luego con los productos.

La forma iterativa simplemente hace:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

Diferencia de rendimiento

Ambos algoritmos (iterativo y recursivo) hacen el mismo proceso (multiplicar los números desde 1 hasta n) pero el método recursivo es menos eficiente por el consumo de memoria y stack.

A menos de que el código se someta a optimizaciones de compilador como la **optimización de llamada de cola** o este escrito en lenguajes funcionales puros como Haskell, el método recursivo posee el riesgo de crear un desborde de pila de llamadas (stack overflow) y consumir tanta memoria como lo diga el número a calcular.

Para el caso específico de Python, este lenguaje no es funcional puro ni ofrece optimizaciones de cola, por lo que se recomienda usar más el enfoque iterativo.

2. K-permutaciones

Para las k-permutaciones podríamos usar los factoriales anteriores, o el de la librería `math`. Sin embargo vamos a indagar de que van las k-permutaciones para hallar una optimización. La fórmula de las k-permutaciones es.

$$nPk = \frac{n!}{(n-k)!}$$

Si desenrollamos el factorial $n!$ hasta llegar a $(n-k)!$ obtenemos:

$$nPk = \frac{n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-k+1) \cdot \cancel{(n-k)!}}{\cancel{(n-k)!}}$$

$$nPk = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-k+1)$$

Esta es de hecho la forma original de la k-permutación, que es una aplicación del principio del producto (o de las cajas) para la elección sucesiva de opciones a la hora de combinar los elementos. Con esta fórmula podemos obtener la k-permutación usando un único bucle de forma más eficiente:

```
## Implementa la k-permutación de un conjunto de n elementos para elegir k elementos
def k_permutation(n: int, k: int) -> int:
    # Error si alguno es menor que cero o si k es menor que n
    if k < 0 or n < 0 or k < n:
        raise ValueError(f"Valores no válidos: {n}, {k}")

    result: int = 1
    for i in range(n - k + 1, n + 1):
        result *= i

    return result
```

Aplicación de consola

Para una aplicación de consola solo usaremos prints e inputs para seguir un flujo de usuario básico. También se incluyen verificaciones de errores bien manejados y código limpio y modular.

Esto estará en el archivo `bono_1_permutaciones/main.py`

Pruebas y casos especiales

Para probar este sistema vamos a probar cuatro situaciones, con tres ejemplos diferentes para cada uno:

1. Números correctos en un rango común (1-30)
2. Uso de negativos y del cero
3. Números gigantes (10^6 y superiores)
4. Rangos equivocados en k-permutación ($k > n$)

Esto estará en el archivo `bono_1_permutaciones/tests.py`

Comentarios y extras

- En algún momento hice un prototipo en una libreta de Jupyter, pero rápidamente se volvía lento y con bugs típicos de las libretas en local. Decidí borrarla.
- También pensé en usar Julia en lugar de Python, pero no lo conozco lo suficientemente bien para usarlo de forma cómoda. Solo como curiosidad, así sería el factorial recursivo allí.

```
function recursive_factorial(n::Int64)::Int64
    if n < 0
        throw(DomainError("Valor no válido: $(n)"))
    end
    return n != 0 ? n * recursive_factorial(n - 1) : 1
end
```

- Uso Python 3.13.5 para este ejercicio. Pero funcionará en cualquier Python moderno.